are implemented behind the scenes. See “Exploring the code behind `IHttpClientFactory` in depth” at http://mng.bz/8NRK.

Rotating handlers with `IHttpClientFactory` solves both the problems we’ve discussed. Another bonus is that it’s easy to replace existing uses of `HttpClient` with `IHttpClientFactory`.

`IHttpClientFactory` is included by default in ASP.NET Core. You simply add it to your application’s services in Program.cs:

```
builder.Services.AddHttpClient();
```

This registers the `IHttpClientFactory` as a singleton in your application, so you can inject it into any other service. The following listing shows how you can replace the `HttpClient` approach from listing 33.2 with a version that uses `IHttpClientFactory`.

Listing 33.3 Using `IHttpClientFactory` to create an `HttpClient`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddHttpClient();                                    ❶

WebApplication app = builder.Build();

app.MapGet("/", async (IHttpClientFactory factory) =>                ❷
{
    HttpClient client = factory.CreateClient();                      ❸

    client.BaseAddress =                                             ❹
        new Uri("https://example.com/rates/");                       ❹

    var response = await client.GetAsync("latest");                  ❺

    response.EnsureSuccessStatusCode();                              ❺
    return await response.Content.ReadAsStringAsync();               ❺
});
app.Run();
```